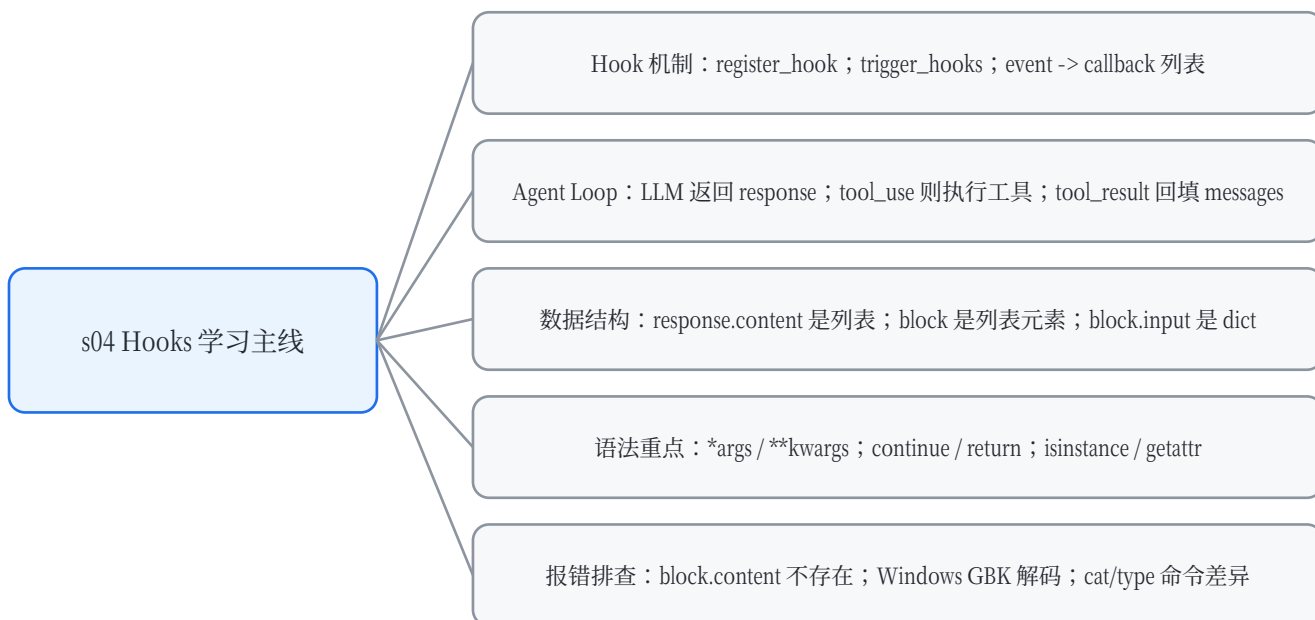


s04 Hooks 对话整理笔记

围绕 Agent 工具调用、Hook 机制、参数解包、response.content/block 结构、调试报错与 Windows 编码问题的系统总结

阅读目标：看懂 s04_hooks/code.py 的主循环；知道 block 从哪里来；理解 PreToolUse/PostToolUse/Stop 的触发时机；能修复本次遇到的 AttributeError、UnicodeDecodeError 和 TypeError。

总览思维导图



目录

1. 本次对话主线与最终结论
2. Hook 系统：注册、触发、拦截
3. Agent Loop 的核心流程
4. response.content 与 block 的来源
5. *args / **kwargs / **block.input
6. summary_hook、isinstance、getattr 与 messages 层级
7. Stop / PreToolUse / PostToolUse 的 continue 逻辑
8. 本次报错排查与修复方案
9. 我的代码与示例代码差异
10. 局部变量、可变对象与函数作用域
11. 最终推荐代码片段与检查清单

1. 本次对话主线与最终结论

这次对话围绕 s04_hooks 版本的 Agent 展开：你先理解了 register_hook / trigger_hooks 的基本结构，再逐步追问 block 从哪里来、response.content 为什么能遍历、hook 参数为什么不一样、continue 跳到哪里、summary_hook 如何统计工具调用，最后排查了运行 README.md 时遇到的 Windows 编码报错。

核心结论：Hook 系统不是 Python 自动理解大模型文本，而是模型 API 返回了结构化 content block；Python 只是遍历 response.content 这个列表。工具调用前后通过 trigger_hooks(event, *args) 把 block、output 等对象传给注册好的 callback。

问题	最终理解
block 从哪来？	来自 for block in response.content，是 response.content 列表中的单个内容块。
block 的属性属于谁？	type/name/input/id 属于单个 block，不属于 response.content 这个列表本身。
谁把文本切成 block？	不是 Python 切的，是模型 API 协议返回结构化 JSON，SDK 再转成 Python 对象。
**block 是解包吗？	代码里真正解包的是 **block.input；block 是对象，block.input 才是 dict。
PostToolUse 为什么传两个参数？	因为工具执行后既要知道哪个工具 block，也要知道工具输出 output。
本次报错根因？	block.content 属性不存在；Windows 默认 GBK 解码 README 或命令输出失败。

2. Hook 系统：注册、触发、拦截

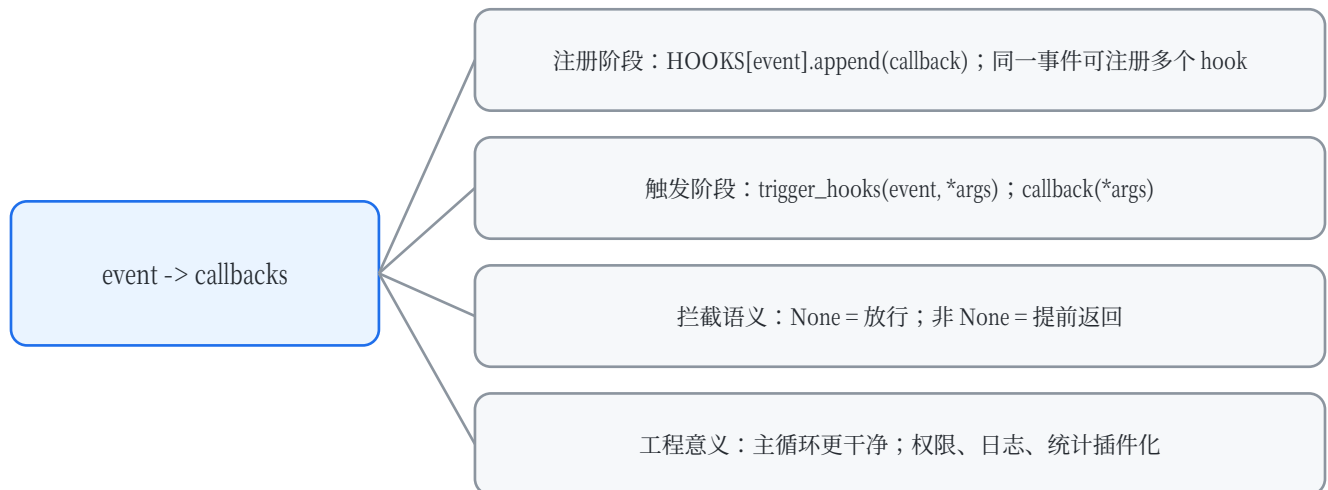
Hook 可以理解为“在程序运行到某些关键节点时，自动调用提前注册的函数”。本项目中事件名包括 UserPromptSubmit、PreToolUse、PostToolUse、Stop。

```
# Hook 注册与触发的核心代码
HOOKS = {"UserPromptSubmit": [], "PreToolUse": [], "PostToolUse": [], "Stop": []}

def register_hook(event: str, callback):
    HOOKS[event].append(callback)

def trigger_hooks(event: str, *args):
    for callback in HOOKS[event]:
        result = callback(*args)
        if result is not None:
            return result
    return None
```

register_hook(event, callback)	把 callback 函数追加到 HOOKS[event] 对应的列表中。
trigger_hooks(event, *args)	遍历 HOOKS[event] 中的所有 callback，并用 callback(*args) 调用。
return None	表示当前 hook 没有拦截，流程继续。
return 非 None	表示 hook 触发了拦截或强制结果，trigger_hooks 会立刻返回，不再执行后续 callback。



3. Agent Loop 的核心流程

s04 的主循环和 s03 的核心差异是：s03 在 loop 里硬编码权限检查；s04 把权限检查、日志、输出检查、停止总结都移动到 hook 函数里。主循环只在合适位置触发事件。

```
# Agent Loop 精简结构
def agent_loop(messages: list):
    while True:
        response = client.messages.create(...)
        messages.append({"role": "assistant", "content": response.content})

        if response.stop_reason != "tool_use":
            force = trigger_hooks("Stop", messages)
            if force:
                messages.append({"role": "user", "content": force})
                continue
            return

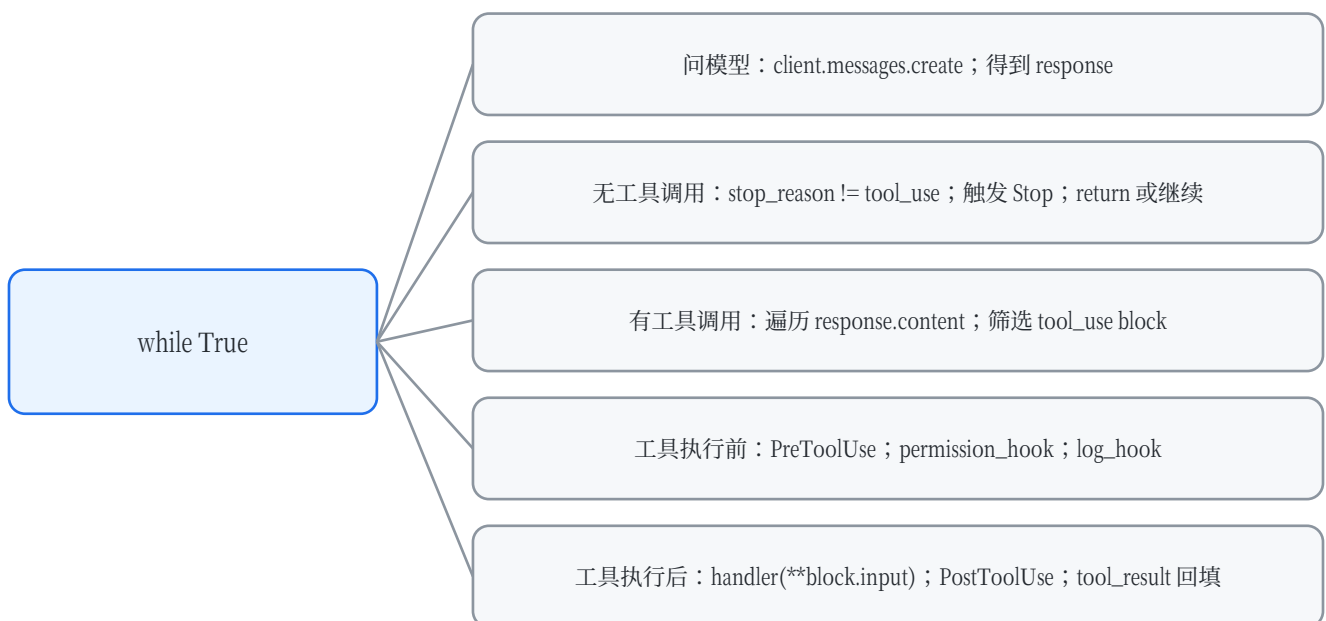
        results = []
        for block in response.content:
            if block.type != "tool_use":
                continue

            blocked = trigger_hooks("PreToolUse", block)
            if blocked:
                results.append({"type": "tool_result", "tool_use_id": block.id,
                               "content": str(blocked)})
                continue

            handler = TOOL_HANDLERS.get(block.name)
            output = handler(**block.input) if handler else f"Unknown: {block.name}"
            trigger_hooks("PostToolUse", block, output)
            results.append({"type": "tool_result", "tool_use_id": block.id, "content":
                           output})

        messages.append({"role": "user", "content": results})
```

Agent Loop 流程图



4. response.content 与 block 的来源

response.content 不是普通字符串，而是模型 API 返回的结构化内容块列表。Python 的 for 循环只是普通列表遍历；真正生成 text/tool_use block 的是模型 API 协议和 SDK。

```
# response.content 的结构化来源
# 底层可以理解成类似这样的结构化 JSON：
{
  "stop_reason": "tool_use",
  "content": [
    {"type": "text", "text": "我需要读取文件。"},
    {"type": "tool_use", "id": "toolu_123", "name": "read_file",
      "input": {"path": "README.md"}}
  ]
}

# SDK 转成对象后，可以写：
for block in response.content:
    print(block.type)
    print(block.name)
    print(block.input)
```

注意：response.content 是列表；block 是列表里的单个元素；block.type / block.name / block.input / block.id 是单个 block 的属性。不能把它们理解成 response.content 这个列表本身的属性。

5. *args / **kwargs / **block.input

这次对话中你特别区分了 *args 与 **kwargs。两者都可以和“解包”有关，但对象不同：* 用于位置参数，** 用于关键字参数。

写法	出现场景	含义
def f(*args)	函数定义	收集多余的位置参数，args 是 tuple。
f(*args)	函数调用	把 tuple/list 拆成位置参数传入。
def f(**kwargs)	函数定义	收集多余的关键字参数，kwargs 是 dict。
f(**kwargs)	函数调用	把 dict 拆成 key=value 的关键字参数。

```
# *args 在 Hook 系统中的作用
# trigger_hooks 接收任意数量位置参数
trigger_hooks("PostToolUse", block, output)

# 函数内部：
args = (block, output)
callback(*args)

# 等价于：
callback(block, output)

# **block.input 才是关键字解包
# block.input 是工具参数字典
block.input = {"command": "ls"}
handler(**block.input)

# 等价于：
handler(command="ls")
```

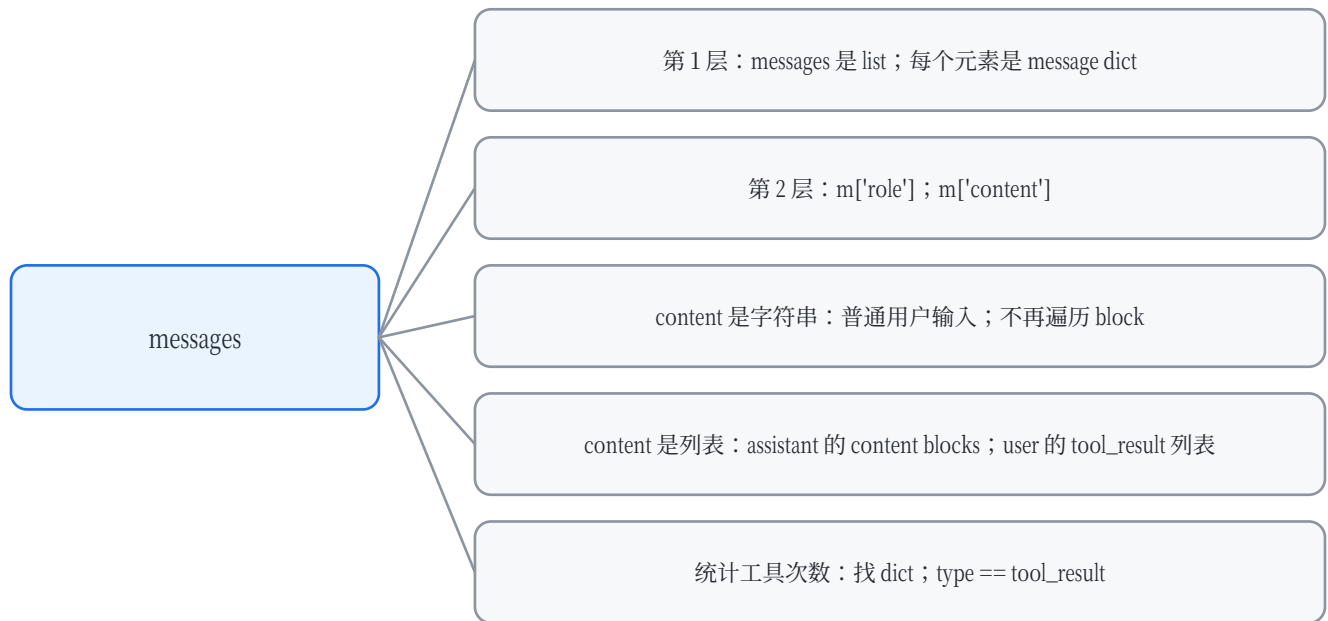
不要写 handler(**block)。block 是 ToolUseBlock 对象，不是 dict；通常会报 TypeError。正确写法是 handler(**block.input)，因为 block.input 才是工具参数字典。

6. summary_hook、isinstance、getattr 与 messages 层级

你提出的 summary_hook 写法基本正确。关键是区分 response.content 和 messages：response.content 直接装 block；messages 是完整对话历史，每条 message 的 content 里才可能有 block 或 tool_result。

```
# 适合统计整个 messages 中 tool_result 的写法
def summary_hook(messages: list):
    tool_counts = 0
    for m in messages:
        content = m.get("content", [])
        if isinstance(content, list):
            for block in content:
                if isinstance(block, dict) and block.get("type") == "tool_result":
                    tool_counts += 1
    print(f"[HOOK] Summary: {tool_counts}")
    return None
```

isinstance(block, dict)	判断 block 是否是字典。只有确认是 dict，才能安全使用 block.get(...).
block.get('type')	适用于 dict，例如 {'type': 'tool_result'}。
getattr(block, 'type', None)	适用于 SDK 对象，例如 ToolUseBlock(type='tool_use')。
tool_use	模型请求调用工具。
tool_result	程序把工具执行结果返回给模型。统计工具执行次数时通常统计 tool_result。



7. Stop / PreToolUse / PostToolUse 的 continue 逻辑

7.1 Stop hook 中的 continue

```
# 模型没有工具调用时的停止逻辑
if response.stop_reason != "tool_use":
    force = trigger_hooks("Stop", messages)
    if force:
        messages.append({"role": "user", "content": force})
        continue
    return
```

continue 不是控制 if 的语句，而是控制最近一层循环的语句。这里最近的循环是 while True，所以 continue 会结束当前轮循环，回到 while True 开头，再次请求模型。return 则直接退出 agent_loop 函数。

语句	作用范围	在这里的效果
continue	最近一层循环	跳回 while True 开头，继续下一轮 Agent Loop。
return	当前函数	退出 agent_loop。
if	条件分支	本身不是循环，continue 不会只作用于 if。

7.2 PreToolUse 被拦截时的 continue

```
# 工具调用前被 hook 拦截
blocked = trigger_hooks("PreToolUse", block)
if blocked:
    results.append({"type": "tool_result", "tool_use_id": block.id,
                  "content": str(blocked)})
    continue
```

这里的 continue 作用于 for block in response.content。它表示：当前这个工具调用已经被拒绝了，不执行 handler(**block.input)，直接处理下一个 block。即使工具未真正执行，也要返回一个 tool_result，告诉模型这个工具调用被拒绝了。

更严谨写法：if blocked is not None: 因为 trigger_hooks 的拦截语义是“非 None 即拦截”。如果 hook 返回空字符串，if blocked 会误判为 False。

7.3 PostToolUse 为什么传 block, output

```
# PostToolUse 的参数约定
output = handler(**block.input)
trigger_hooks("PostToolUse", block, output)

# 对应 hook 函数应该写成：
def large_output_hook(block, output):
    ...
```

PreToolUse 发生在工具执行前，只有 block；PostToolUse 发生在工具执行后，此时既有工具调用请求 block，也有工具输出 output，所以要传两个参数。

8. 本次报错排查与修复方案

8.1 AttributeError: ToolUseBlock 没有 content

```
# 错误写法
def large_output_hook(block, output):
    if len(str(block.content)) > 1000000:
        ...
```

原因：block 是 ToolUseBlock，表示模型请求调用工具。它通常有 id/type/name/input，但没有 content。content 是后面你手动构造的 tool_result dict 里的字段。

```
# 正确写法：检查 output，不检查 block.content
def large_output_hook(block, output):
    output_text = str(output)
    if len(output_text) > 1000000:
        print(f"Output too large from {block.name}: {len(output_text)} bytes")
    return None
```

8.2 UnicodeDecodeError: Windows GBK 解码失败

你的 README.md 或命令输出中含有 GBK 无法解码的字节。Windows 中文环境里 Path.read_text() 和 subprocess.run(text=True) 如果不指定 encoding，容易使用系统默认 GBK，导致 UnicodeDecodeError。

```
# run_read 修复
# read_file 工具建议修复
lines = safe_path(file_path).read_text(
    encoding="utf-8",
    errors="replace"
).splitlines()
```

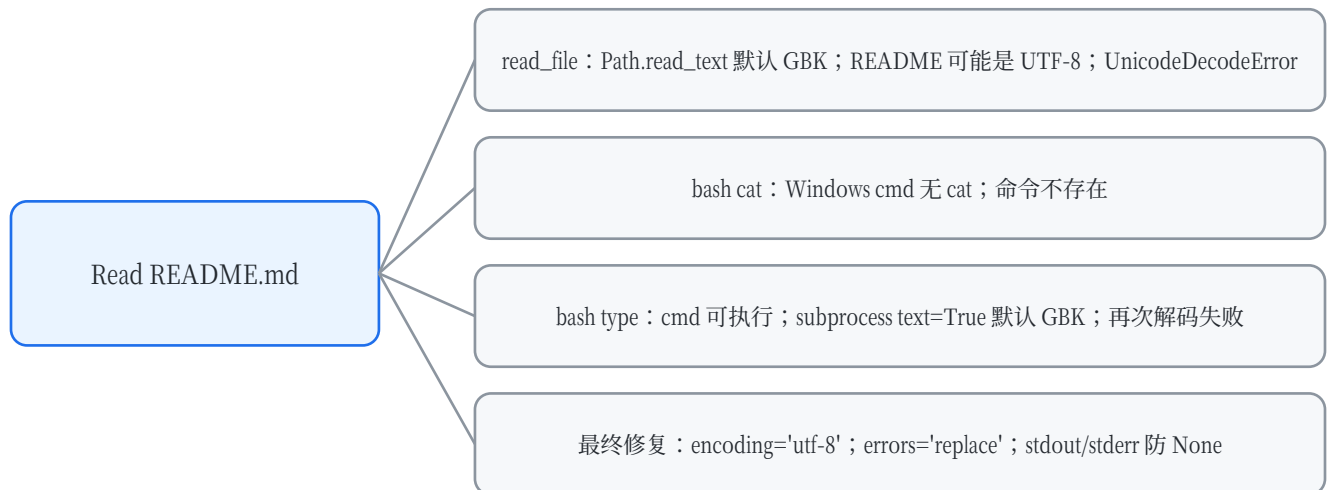
```
# run_bash 修复
# bash 工具建议修复
r = subprocess.run(
    command,
    shell=True,
    cwd=WORKDIR,
    capture_output=True,
    text=True,
    encoding="utf-8",
    errors="replace",
    timeout=120,
)
stdout = r.stdout or ""
stderr = r.stderr or ""
out = (stdout + stderr).strip()
```

GBK 报错后又出现 TypeError: NoneType + str，是因为 stdout 读取线程解码失败，r.stdout 可能变成 None；所以拼接前必须写 stdout = r.stdout or ""。

8.3 cat/type 与 Windows 命令环境

模型先用 read_file 失败后，尝试 bash('cat README.md')。但在 Windows cmd 环境中 cat 不是内置命令，因此出现“cat 不是内部或外部命令”。之后模型尝试 type README.md，这是 Windows cmd 命令，但输出仍然可能被 Python 按 GBK 解码失败。

报错链路思维导图



9. 我的代码与示例代码差异

你看到的输出和示例答案有差异，并不代表 hook 逻辑错。你的输出中已经有 [HOOK] UserPromptSubmit、[HOOK] read_file、[HOOK] bash，说明 Hook 系统正常触发。差异主要来自打印方式、工具参数名、编码处理和命令环境。

差异点	示例代码	你的代码/现象	影响
UserPromptSubmit	主循环里主动 trigger_hooks('UserPromptSubmit', query)	你后来也加了，所以能打印 working in ...	这是用户输入提交前 hook。
工具日志	主要由 log_hook 打印 [HOOK] xxx(...)	你额外打印了 > tool_name 和 \$ block.input。	你的输出更详细。
read_file 参数	示例用 path。	你的工具 schema 用 file_path。	只要 handler 形参一致就能运行；权限检查要兼容 file_path。
large_output_hook	检查 output。	之前误写 block.content。	导致 AttributeError。
run_read/run_bash	原示例也没有完整处理 Windows 编码。	你在 Windows 下读 UTF-8 README 报 GBK 错。	需要指定 encoding/errors。

重点：你的 Hook 打印是正常的。read_file 失败后模型继续尝试 bash，这是 Agent 的工具重试行为；不是 Hook 多执行了，也不是循环错了。

10. 局部变量、可变对象与函数作用域

你最后确认了变量作用域：函数内部定义的局部变量一般不受外界同名变量影响。但如果外部传入的是 list/dict 这类可变对象，函数内部可以修改这个对象本身。

```
# 局部变量不会改外部同名变量
x = 10

def func():
    x = 20
    print(x)

func() # 20
print(x) # 10

# 传入可变对象时，函数内可以修改对象内容
history = []

def agent_loop(message):
    message.append({"role": "user", "content": "hello"})

agent_loop(history)
print(history) # history 被修改了
```

变量名	函数内部的 message 和外部的 history 可以是不同名字。
对象引用	它们可能指向同一个 list 对象。
append 的影响	message.append(...) 会修改同一个 list，所以外部 history 也能看到变化。
重新赋值	如果在函数内 message = []，只是让局部变量指向新列表，不会替换外部 history。

11. 最终推荐代码片段与检查清单

11.1 推荐修复版 run_read

```
def run_read(file_path: str, limit: int | None = None) -> str:
    try:
        lines = safe_path(file_path).read_text(
            encoding="utf-8",
            errors="replace"
        ).splitlines()

        if limit and limit < len(lines):
            lines = lines[:limit]

        return "\n".join(lines)
    except Exception as e:
        return f"Error: {e}"
```

11.2 推荐修复版 run_bash

```
def run_bash(command: str) -> str:
    dangerous = ["rm -rf /", "sudo", "shutdown", "reboot", "> /dev/"]
    if any(d in command for d in dangerous):
        return "Error: Command not allowed."

    try:
        r = subprocess.run(
            command,
            shell=True,
            cwd=WORKDIR,
            capture_output=True,
            text=True,
            encoding="utf-8",
            errors="replace",
            timeout=120,
        )

        stdout = r.stdout or ""
        stderr = r.stderr or ""
        out = (stdout + stderr).strip()
        return out[:50000] if out else "(no output)"

    except subprocess.TimeoutExpired:
        return "Error: Timeout (120s)"
    except Exception as e:
        return f"Error: {type(e).__name__}: {e}"
```

11.3 推荐修复版 large_output_hook

```
def large_output_hook(block, output):
    """PostToolUse: 检查输出是否过大。"""
    output_text = str(output)
    if len(output_text) > 1000000:
        print(
            f"\n\033[33m[HOOK] Output too large from {block.name}:"
            f"\033[0m"
        )
    return None
```

11.4 最终检查清单

检查项	判断标准
Hook 注册	register_hook('PreToolUse', permission_hook) 和 register_hook('PreToolUse', log_hook) 是否都执行。

Hook 触发	agent_loop 内是否有 trigger_hooks('PreToolUse', block)、trigger_hooks('PostToolUse', block, output)。
UserPromptSubmit	主输入循环里是否调用 trigger_hooks('UserPromptSubmit', query)。
参数名一致	TOOLS 的 input_schema 字段名要和 handler 函数形参一致，例如 file_path vs path。
路径检查	permission_hook 中写文件路径检查要兼容 block.input.get('file_path') 或 block.input.get('path')。
编码安全	所有 read_text/subprocess 输出都指定 encoding='utf-8', errors='replace'。
统计逻辑	统计整个 messages 时先遍历 message，再进入 content；统计 response.content 时可以直接遍历 block。

一句话总复习：s04 Hooks 的本质是把 Agent Loop 中的扩展逻辑插件化。主循环只负责在关键节点触发事件；权限检查、日志、输出检查、停止总结都交给注册的 hook 函数完成。